

Hybrid Recipe Search and Personalized Recommendation System

DSCI 641: Recommender Systems for Data Science

Ian Auger

Abstract

This report describes an end-to-end personalized recipe recommendation system developed incrementally across four graduate courses. The system integrates a PySpark sentiment-tagging pipeline, a residual neural network embedding model (RecipeNet), an Elasticsearch-based IR system, and a two-stage personalized recommender evaluated on the Food.com corpus using leave-N-out over 200 users. The central finding is that the RecipeNet embedding space and the Food.com taxonomy tag space are orthogonal: embeddings encode recipe quality while tags encode culinary identity. The two-stage architecture uses each signal where it is effective and achieves TagSim@10 = 0.186, a 79% improvement over raw embedding KNN and a 96% improvement over a cold-start quality baseline.

1. Introduction

Recipe search has an unusual three-way constraint: user intent is simultaneously culinary (what kind of food?), practical (how much time, how much skill?), and personal (what does this specific user like?). A query for "easy chicken dinner" from a user with a history of Mexican-Southwestern recipes should produce different results than the same query from someone whose reviews skew toward quick French bistro-style dishes, even when BM25 text matching returns identical output for both. A user who mostly gives 5-star ratings to bold, spice-heavy dishes and consistently flags recipes as "too bland" should see a meaningfully different shortlist than one who loves mild weeknight food.

This project builds a system that handles that constraint, developed incrementally across four courses with each phase producing standalone artifacts that compose into a single inference stack. Phase 1 developed a PySpark sentiment tagging pipeline, applying 17 culinary labels per review. Phase 2 built a deep learning residual neural network called RecipeNet which produced 128D recipe embeddings and Bayesian quality predictions. Phase 3 built an Information Retrieval system using BM25 with a SemanticReranker system with 5-mode ablation. Then this final phase built a two-stage personalized recommender using tag-filtered candidate generation with embedding KNN ranking. The central question: given a user's Food.com review history, can we generate recipe recommendations that better match their demonstrated culinary preferences than a quality-based ranking baseline?

2. Data

The Food.com corpus (Majumder et al., 2019) contains 231,637 recipes and 1,132,367 user interactions spanning 2000 to 2018. The main data quality challenge is rating positivity skew: 77% of ratings are 4 or 5 stars, which makes raw ratings a poor training target for any quality model. After Phase 1 processing, approximately 65K reviews retain enough text for tag

labeling, 173K recipes have embeddings and are indexed in Elasticsearch, and about 12K users have at least 10 reviews. Evaluation uses a subset of 200 users with at least 15 reviews to ensure meaningful training signal after the 5-review holdout.

3. System

The four phases share a common artifact format that enables clean composition without retraining. Phase 1's labeled review parquet feeds both Phase 2 model training and Phase 4 user profile construction. Phase 2's embedding checkpoint is loaded by both Phase 3's SemanticReranker and Phase 4's ranking stage. Phase 3's Elasticsearch index and search engine are reused as the Phase 4 backbone unchanged.

3.1 Phase 1: Sentiment Tagging (DSCI 632)

The Phase 1 pipeline converts review text into a structured 17-tag signal using a PySpark Word2Vec classifier. Tags are assigned by cosine distance to learned centroid embeddings, with calibrated per-tag thresholds. Each review receives a boolean prediction and a continuous intensity score (distance to centroid) for each of 17 tags organized into positive, negative, and neutral categories:

Polarity	Tags
Positive	family_hit, delicious_tasty, would_make_again, easy_quick, crispy_crunchy, moist_tender
Negative	ingredient_issue, would_not_make_again, too_spicy, time_consuming_complex, bland_lacks_flavor, dry, too_sweet, mushy_soggy, too_acidic, too_salty
Neutral	substitution_modification

In Phase 4, these tags become the primary user-preference signal. For each user, we build a 17-dimensional affinity vector by taking a weighted mean of tag intensities across their review history: weights combine the rating deviation from the global mean with an exponential recency factor (180-day half-life), so recent strong opinions count most. The top-40 tags by affinity score then drive Stage 1 candidate retrieval in the personalized recommender.

3.2 Phase 2: RecipeNet Embedding Model (CS 615)

RecipeNet is a dual-encoder residual MLP trained to predict Bayesian-smoothed recipe quality from two input streams: recipe metadata (cooking time, step count, nutritional features) and aggregated Phase 1 tag signals. Because Food.com ratings are heavily skewed positive, training targets are smoothed by pulling each recipe's mean rating toward the global mean in proportion to review count, reducing volatility for sparse recipes. The 128-dimensional bottleneck layer is extracted as the recipe embedding. Six architectures were tested under identical conditions. RMSE differences are less than 0.003 across all models, so selection was driven by embedding geometry. Deep architectures without residual connections collapsed the latent space toward a near-1D quality axis;

RESIDUAL_V2 preserves a smooth two-dimensional manifold with coherent local structure, verified via UMAP:

Architecture	Loss	RMSE	Embedding geometry
Deep MLP	MSE	0.2964	Collapsed (near-1D)
Shallow MLP	Huber	0.2964	Moderate
Residual V2 (selected)	MSE	0.2973	Smooth, coherent manifold
Residual	Huber	0.2975	Good
Two-Tower	Huber	0.2980	Good
Residual V3	Huber	0.2997	Good

RecipeNet was intentionally trained to predict Bayesian-smoothed recipe quality rather than culinary identity: in Phase 3, a quality signal is exactly what the SemanticReranker needs to differentiate between structurally similar BM25 candidates. High-quality recipe embeddings do cluster tightly in cosine space, which works well when the goal is ranking comparable results. In Phase 4, however, that same property becomes a constraint: raw KNN over the full 173K corpus returns a quality shortlist rather than a culinarily matched one, because the embedding space was never designed to separate recipes by genre. This tradeoff directly motivates the two-stage architecture in Phase 4.

3.3 Phase 3: Information Retrieval System (INFO 624)

The IR system uses Elasticsearch BM25 for first-stage candidate retrieval, parsing queries into structured slots (dietary restrictions, protein type, cuisine, cooking method, time bounds). A SemanticReranker second stage combines lexical, alignment, semantic, and quality signals with stratified weights that adapt to query intent richness. Queries with many structured slots rely more on alignment; sparse queries lean on semantic similarity. This prevents fixed-weight schemes from degrading when the query signal is minimal.

In Phase 4, the engine accepts a user signal parameter. An alpha blend coefficient controls the mix between query-driven and user-driven ranking. At alpha=0, ranking is driven entirely by the user's taste centroid; at alpha=1, it behaves as standard query search with no personalization.

3.4 Phase 4: Personalized Recommender (DSCI 641)

Phase 3 retrieves well but is user-agnostic. Phase 4 adds personalization from review history without requiring live accounts or real-time interaction data.

For each user, a profile is constructed from their gold-labeled reviews. The embedding centroid is a rating-and-recency weighted mean of recipe embeddings, where the recency weight decays exponentially with a 180-day half-life and reviews at 3 stars or below have their deviation amplified 2x to sharpen the negative signal. The tag affinity vector summarizes the

user's 17-dimensional culinary preference across all reviews under the same weighting scheme.

The pipeline runs in two stages. Stage 1 (CandidateGenerator) scores all recipes by IDF-weighted tag recall against the user's top-40 history tags, generating a pool of 1,000 culinarily coherent candidates. Stage 2 (Personalized SemanticReranker) runs cosine KNN within that pool, boosted by the 17D affinity vector, returning top-k results. Three interaction modes are supported:

Mode	Stage 1 signal	Stage 2 signal	Entry point
Cold-start	(none)	Quality ranking from full bundle	New user
Session-based	Union of seed recipe tags	Mean seed embedding	Search + Add to session
History-aware	Top-40 user history tags	Rating x recency centroid + affinity boost	User History mode

4. Evaluation

4.1 Setup and Metrics

Evaluation uses leave-N-out holdout (n_holdout=5, n_users=200, min_reviews=15). For each user, the 5 most recent reviews are held out and the recommender generates top-k recommendations from the remaining history. Six conditions are compared:

Condition	Description
cold_start	Quality ranking only, no personalization (baseline)
knn_only	Full-corpus embedding KNN (degenerate baseline)
two_stage (tag)	Tag recall Stage 1 + embedding KNN Stage 2 (primary system)
two_stage (ingr)	Ingredient TF-IDF Stage 1 + embedding Stage 2
two_stage (hybrid)	Tag + ingredient union Stage 1 + embedding Stage 2
unified	Elasticsearch full-text Stage 1 + SemanticReranker (upper bound)

The primary metric is TagSim@k: the mean cosine similarity between recommended recipe taxonomy tags and the union of held-out recipe taxonomy tags across users. TagSim is appropriate because exact item recovery at reasonable list lengths is nearly impossible given the corpus size. A recommendation in the right culinary genre represents a meaningful personalization signal even if it is not the exact held-out item. Secondary metrics include HR@10 (exact item recovery rate), MRR (mean reciprocal rank of the first held-out item), and Stage 1 pool recall (fraction of held-out items appearing in the 1,000-item candidate pool).

4.2 Results

Condition	TagSim@10	HR@10	MRR	Pool Recall
cold_start	0.095	--	--	--
knn_only	0.104	0.005	0.0013	1.000
two_stage (tag)	0.186	0.005	0.0018	0.120
two_stage (ingr)	0.113	0.015	0.0061	0.090
two_stage (hybrid)	0.155	0.010	0.0040	0.195
unified (ceiling)	0.206	0.000	0.0001	0.045

The two-stage (tag) condition achieves TagSim@10 = 0.186, a 79% improvement over knn_only (0.104) and a 96% improvement over cold_start (0.095). The ingredient-based variant recovers exact held-out items at 3x the rate (HR@10 = 0.015 vs. 0.005) but at the cost of 40% lower TagSim, confirming that the two signals measure fundamentally different things. The unified ceiling (0.206) is included as an upper bound rather than a fair comparison: it indexes full recipe text and is not tunable toward the personalized two-stage design. Pool recall for knn_only is 1.0 by construction (the full corpus is the candidate pool), which makes its low TagSim especially telling.

5. Ablation Study

Phase 4 development was incremental: one change at a time, measuring TagSim@10 to isolate each signal's contribution. The progression from the raw KNN baseline to the final system:

Step	Change	TagSim@10	Delta
Baseline	Raw knn_only	0.104	--
+tag Stage 1	Jaccard scoring	0.130	+0.026

+recall scoring	Intersection / IDF sum	0.144	+0.014
+pool size	500 to 1,000 candidates	0.149	+0.005
+recency decay	365-day to 180-day half-life	0.162	+0.013
+negative signal	3-star threshold, 2x amplification	0.170	+0.008
+top_tags 20 to 40	More history tags in Stage 1 query	0.178	+0.008
+quality=0 (final)	Remove quality weight from ranking	0.186	+0.008

Several changes that seemed promising made things worse:

Change tested	Actual result	Explanation
Inject embedding NN into Stage 1	TagSim -40% (0.186 to 0.109)	Embedding similarity is not culinary similarity
Widen Stage 2 pre-filter (3x to 20x)	Recall drops, TagSim unchanged	Tag-similar candidates already survive; widening adds noise
top_tags 40 to 50	+0.001 TagSim, pool recall -12.5%	Borderline-affinity tags dilute Stage 1 retrieval
Quality weight 0.10	TagSim drops	Quality signal conflicts with culinary identity signal

The most counterintuitive result is the embedding injection failure. Adding embedding nearest neighbors to Stage 1 collapsed TagSim back to KNN level. This makes the orthogonality result concrete: embedding-similar recipes cluster around quality, not cuisine, so seeding the candidate pool with them undoes the genre filtering that tag recall provides.

6. Discussion

The tag-based first stage is not a performance optimization; it is a structural necessity. Without it, the embedding-based second stage has no culinarily coherent pool to rank within, and results degrade to a quality shortlist regardless of user history. This is the clearest demonstration of the project's core thesis: personalization is a retrieval problem before it is a ranking problem, and the embedding space cannot solve the retrieval problem because it was not trained to.

Ingredient TF-IDF and taxonomy tags measure genuinely different things. Ingredients recover the exact held-out items at 3x better HR and 3.4x better MRR, while taxonomy tags produce 86% better TagSim. Ingredients are a specificity signal; tags are a genre signal.

Recency and negative signals have outsized impact at Food.com's timescale. Reviews span 18 years, and a recipe from 2003 carries almost no signal about current preferences. Reducing the recency half-life from 365 to 180 days and amplifying low-star deviations produced two of the largest single-step ablation gains. The general lesson: when building user profiles from long interaction histories, recent negative feedback has a much higher signal-to-noise ratio than old positive feedback.

The 10% gap between the two-stage system (0.186) and the unified ceiling (0.206) is structural, not tunable. ES BM25 indexes full recipe text including names, descriptions, ingredient lists, and tag strings, which is a far richer retrieval signal than curated taxonomy tag overlap. A better path forward would be training an embedding model directly on culinary similarity rather than rating prediction, using recipe category pairs as supervision.

7. Limitations

The core measurement challenge is that TagSim and offline leave-N-out evaluation are well-matched to each other but both measure the same thing: culinary genre alignment. The system is explicitly optimized to improve TagSim, so strong TagSim results confirm the architecture works as designed, not necessarily that users would find the recommendations useful. A real user study is the only way to close that gap, and without one it is difficult to say whether a 79% TagSim improvement translates into 79% better recipes on someone's dinner table.

The modeling constraints are more tractable. RecipeNet embeddings are frozen from Phase 2 training on a quality objective, which is the root cause of the orthogonality problem the entire two-stage architecture exists to work around. Retraining on a culinary similarity objective would likely improve both Stage 1 recall and Stage 2 ranking quality in a single change. The rating positivity skew in Food.com also limits how much signal the affinity vector can extract from positive reviews, since most 5-star deviations are small. And the 15-review minimum for history-aware mode is a real cold-start boundary that the session-based fallback only partially addresses.

8. Conclusions

This project and course highlighted the critical necessity of evaluating recommender systems through the lens of specific user constraints rather than accuracy metrics. Building this system clarified that raw algorithmic performance is secondary to signal alignment; a high-performing quality model is effectively useless for personalization if its latent space does not encode the culinary dimensions that actually drive user preference. Learning to bridge this gap through multi-stage architectures and weighted recency decays provided a practical framework for handling the messy, time-variant nature of real-world interaction data.

The multi-course development arc fundamentally shifted my perspective on ML systems from a model-centric view to an artifact-centric one. By building the system incrementally, I realized that the durability of an ML stack depends on the clean composition of its components rather than the complexity of a single model. Successive courses demonstrated that well-defined artifact formats like the recipe embeddings and sentiment tags used here

enable rapid iteration and experimentation across different course domains without requiring a total rework of the underlying infrastructure. For future offerings, incorporating more explicit supervision of the embedding geometry via contrastive learning could further bridge the gap between quality and identity signals identified in the ablation study.

More than any other course in the MSDS program, this one maintained a consistently user-centric viewpoint that kept the work grounded in real-world constraints rather than benchmark optimization. It will be my last course in the program, but the personalization-first mindset it reinforced is one I plan to carry into what comes next. Thank you for a great course and have a good summer.